

Kabuk Nasıl Çalışır ?

Bu bölümde kabuğun nasıl çalıştığından bahsediyor olacağız. Lütfen bu bölümü dikkatinizi vererek tamamlayın. Eğer dikkatlice okuyup uygulamazsanız pek çok hatayla karşılaşmanın yanı sıra eğitimin devamından alacağınız verimi de düşürmüş olursunuz.

Ben bu eğitimde kabuğun çalışma yapısının bazı detaylarını atlıyor olacağım çünkü bazı konulardan bu eğitimde bahsetsek bile henüz Linux sistemini yani tanımaya başladığımız için yani gereken altyapıya henüz sahip olmadığımız için ek detaylar anlaşılır olmayacak. Bu doğrultuda odak noktamızı yani Linux sistem yönetiminin temellerini öğrenme hedefimizi şaşırmadan devam etmeye çalışalım. Eğitime devam ettikçe zaten kabukla ilgili pek çok yeni bilgiyi parça parça öğreniyor olacağız. Yani öğrenmek için acele edip konuları sıkıştırmanın bir anlamı yok. Gerekli olan temel bilgi altyapısı için biz tüm konuları sindire sindire ilerlemeye çalışalım.

Anlatımlarımıza öncelikle kabuğun bizim girdiğimiz komutları nasıl algıladığından bahsederek başlayabiliriz. Bu konuyla başlıyoruz çünkü kabuğun bizi nasıl anladığını bilmek, kabuğa anlayacağı türden komutlar girebilmemiz için çok önemli.

Örnek olması için sistem üzerinde dosyaları ve klasörleri bulma konusunda bize yardımcı olan `find` aracını kullanarak `bash` kabuğunun bizim girdiğimiz komutları nasıl ele aldığından çok kısaca bahsetmek istiyorum. Biliyorum henüz hiç bir komutu ele almadık ve tabii ki `find` komutunu da bilmiyoruz fakat siz benim örnek için kullandığım bu komutun ne olduğuna şimdilik takılmayın. Ben burada `bash` kabuğuna girdiğimiz komutların nasıl çalıştırıldıklarından bahsetmek istiyorum.

Bulunmasını istediğimiz dosyaları kendimiz oluşturabiliriz. Bunun için grafiksel arayüzü de kullanabiliriz fakat ben komut satırından kolayca oluşturmak için `touch ~/test.txt ~/Desktop/test.txt` komutunu giriyorum. Bu komut sayesinde kendi ev dizinimde ve ev dizimin altındaki Desktop klasörü içinde "test.txt" isimli birer dosya oluşturulmuş olacak. Komutu anlamasanız bile şimdilik dosyaları oluşturmak için kopyala yapıştır şekilde kullanabilirsiniz.

Şimdi `find` aracını kullanarak bu dosyaların nasıl bulunabileceğinden bahsederken, `bash` kabuğunun çalışma yapısını ele almaya çalışalım.

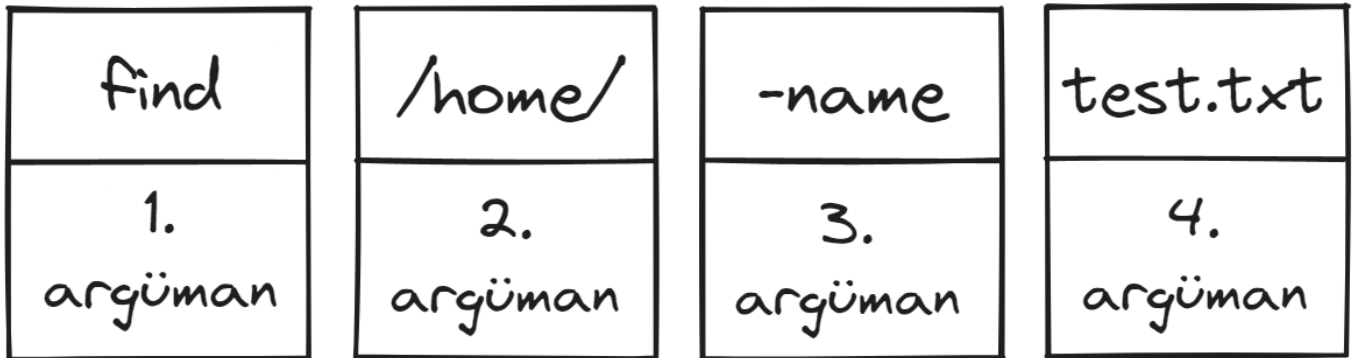
Örnek senaryomuz için diyelim ki benim ev dizinimde birçok dosya ve klasör bulunuyor ve ben de dosya ismi "test.txt" olan dosyalar burada mevcut mu varsa tam olarak hangi dizinde yer alıyorlar diye öğrenmek istiyorum. İşte bu örnek senaryomuz için `find` aracını kullanabiliriz. Ben kendi ev dizinimdeki test isimli tüm dosyaların bulunması için `find /home/ -name test.txt` komutunu giriyorum.

```
(taylan@linuxdersleri) - [~]  
$ find /home/ -name test.txt  
/home/taylan/test.txt  
/home/taylan/Desktop/test.txt
```

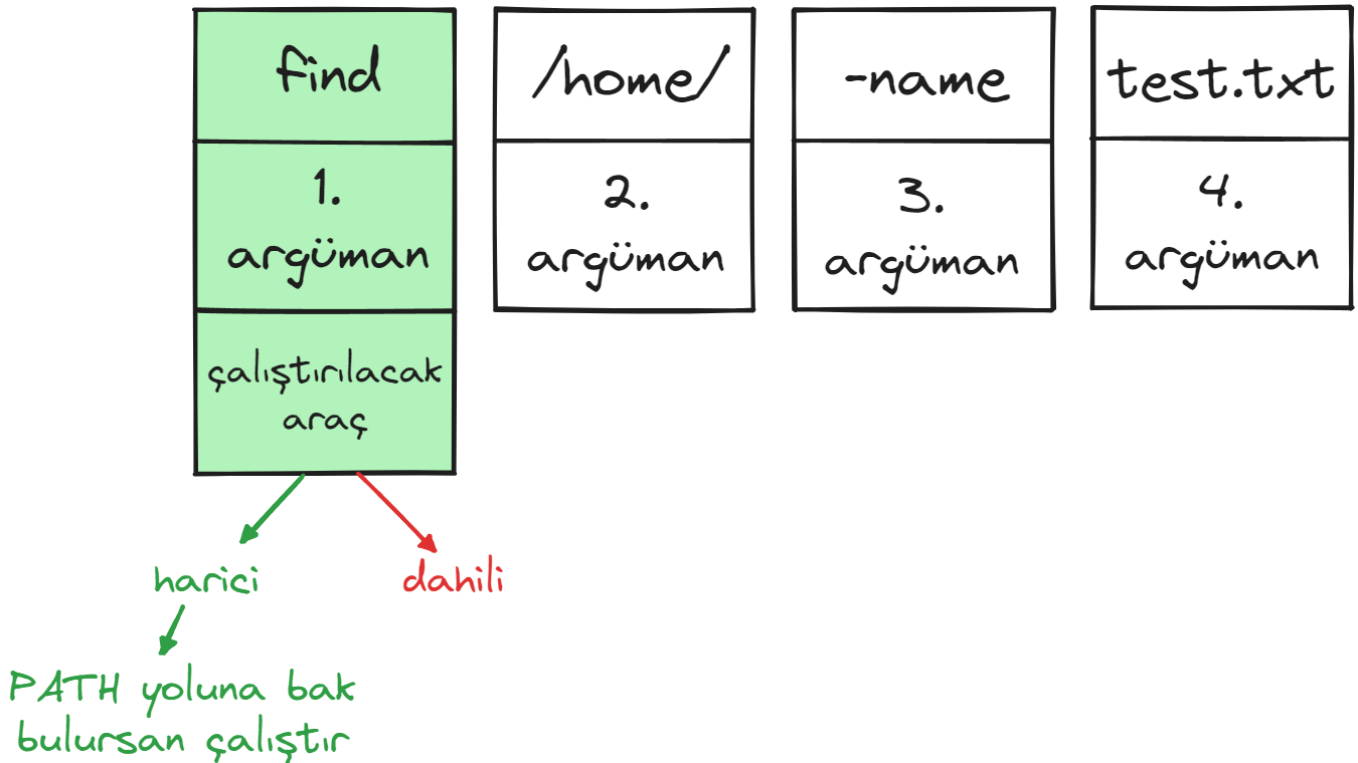
Bakın `find` aracı tam da istediğim şekilde "**test.txt**" dosyaları bulundu ve konsola çıktı olarak bastırıldı. Peki ama bu tam olarak nasıl gerçekleşti ? Yani `bash` kabuğu benim yapmak istediğim işlemi nasıl anladı ve doğru aracı bulup doğru şekilde çalıştırabildi ?

Öncelikle bash kabuğunun bizim girdiğimiz komutu nasıl doğru şekilde anlamlandırabildiğinden kısaca bahsetmemiz gerekirse;

Kabuğa `find /home/ -name test.txt` komutunu girdik. Kabuk öncelikle arasında boşluk bulunan tüm ifadeleri argümanlar olarak ayırdı.



İlk girilen komut yani ilk argüman **find** olduğu için kabuk "find" ifadesini çalıştırılacak araç olarak kabul ediliyor. Ve bu aracı çalıştırmak için de tabii ki önce bu aracı bulması gerekiyor. Bunun için de ilk olarak bu isimle eşleşen dahili bir araç var mı diye bakıyor. Hatırlıyorsanız eğitimin başında kabukların dahili araçları bulunduğu bahsetmiştik. İşte kabuk bir aracı çalıştırmadan önce kendisine verilmiş olan komuttaki ilk argümanın, dahili bir aracın ismi olup olmadığını kontrol ediyor. find aracı bash kabuğunda dahili bir araç olmadığı için elbette herhangi bir eşleşme olmuyor. find komutu dahili bir komut olmadığı için kabuk bu kez PATH olarak geçen birtakım dizinlerin içinde find ismiyle eşleşen çalıştırılabilir bir dosya var mı diye bakıyor. Ve neticede bu dizinlerin birinde find aracının dosyası bulunduğu için bu dosya kabuk tarafından çalıştırıyor.



Çalıştırılacak araç bulunduktan sonra burada yer alan ilk argümandan sonraki argümanlar, istisnai durumlar dışında çalıştırılan araca verilecek argümanlar olarak kabul ediliyor. Dolayısıyla find argümanından ardından yazılmış olan argümanlar find aracının çalışma şeklini tanımlamak için bulunuyor. Örneğin ikinci argüman find aracının nerede araştırma yapması yani nereye bakması gerektiğini belirten bir **parametredir**. Buradaki

argümana “**parametre**” diyorum çünkü bu argüman find aracının spesifik olarak nereye bakması gerektiğini haber veren bir bilgi bulunduruyor. Buradaki “**/home/taylan/**” dizini yerine başka herhangi bir dizin de belirtilebilir. Yani buradaki argüman aslında çalıştırılan araç için bir parametre.

Üçüncü argüman ise find aracına hangi tipte veri araması gerektiğini belirtmemizi sağlayan **seçenektir**. Seçenekler yazılırken tıpkı bu komutumuzda da olduğu gibi genellikle başında tek ya da çift kısa çizgi bulunduruyor. Tek ya da çift kısa çizgi olması tamamen kullandığınız aracın seçenekleri nasıl kabul ettiğine bağlı. Çünkü seçenekler aslında zaten araç geliştiricileri tarafından aracın çeşitli özelliklerinin kullanılabilmesi için önceden tanımlanmış bazı özel ifadelerdir. İleride farklı araçları ele aldığımızda seçeneklerin başında tek veya çift kısa çizgi bulunabildiğini bizzat görmüş olacaksınız zaten.

Burada “**-name**” **seçeneğinin** ardından girdiğimiz “**test**” **argümanı** da aranacak dosya veya klasörün ismini belirten parametredir. Biz **-name** seçeneği sayesinde find aracına “**test.txt**” ismini araştırmasını özellikle belirtebiliyoruz.

find	/home/	-name	test.txt
1. argüman	2. argüman	3. argüman	4. argüman
çalıştırılacak araç	parametre	seçenek	parametre

En nihayetinde doğru şekilde kullandığımız tüm seçenek ve parametrelerle birlikte find aracına istediğimiz görevi yazılı şekilde iletip, işin yerine getirilmesini sağlayabiliyoruz.

Tekrar özetleyecek olursak: kabuk find aracını bulup çalıştırdı ona buradaki argümanları iletti, find aracı da aldığı argümanları kendisine göre yorumlayıp görevini yerine getirdi. Tüm işleyişin özeti bu.

Elbette kullanılan araca göre seçeneklerin veya parametrelerin çeşidi ve sıralaması farklı olabilir. Çünkü her aracın kendisine verilen argümanları ele alış biçimi farklıdır. Yani aracın yapısına göre girilmesi gereken seçenek ve parametrelerin sıralaması değişebileceği için benim ele aldığım örnekteki sıralamanın ve seçeneklerin doğrudan bir önemi yok, burada önemli olan kavramlardır. **Argüman** ne demek, **seçenek** ne demek, **parametre** ne demek bunları bilmeniz önemli. Çünkü tüm eğitim boyunca bu kavramları kullanarak açıklama yapıyor olacağım. Dolayısıyla benim söylemek istediklerimi doğru şekilde anlayabilmek için bu temel kavramları da biliyor olmanız gerekiyor.

Temel kavramlardan da bahsettiğimize göre anlatım sırasında bahsi geçen **PATH** dizini kavramından bahsederek devam edebiliriz.

PATH Yolu

PATH esasen sistem üzerinde tanımlı olan bir değişkendir. Bu değişken, kabuğun çalıştırılacak dosyaları araması gereken dizin adreslerini tutuyor. Bu adresleri öğrenmek için daha önce varsayılan kabuğumuzu öğrenirken sorguladığımız **SHELL** değişkenine benzer şekilde PATH değişkenini sorgulamak için kabuğa `echo $PATH` komutunu girebiliriz. Buradaki dolar işareti `echo` aracının, **PATH** isimli değişkenin değerini konsola bastırmasını sağlıyor. Bu durumdan daha sonra ayrıca detaylı şekilde bahsedeceğiz. Şimdi aldığımız çıktıya odaklanacak olursak:

```
└─$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games
es:/usr/games
```

Bakın sıralı şekilde bazı dizin adresleri çıktı olarak bastırıldı. Burada gördüğümüz iki nokta işareti ile ayrılmış olan her bir dizin adresi, kabuğun bir aracın çalıştırılabilir dosyasını ararken soldan sağa doğru sırasıyla bakacağı dizinlerin adresidir. İşte sırasıyla bakılan bu dizinlere de PATH yolu deniyor. Kabuk, harici bir komutu hangi dizinlerde arayacağını bu PATH değişkenine bakarak öğreniyor. Dolayısıyla eğer kabuk üzerinden bir aracı çalıştırmak istiyorsanız, aracın çalıştırılabilir dosyası mutlaka PATH değişkeninde tanımlı olan dizinlerden birinde olmalı. Ayrıca dilerseniz, PATH değişkenine yeni dizin adresleri ekleyerek, kabuğun bakması gereken dizinleri de çoğaltabilirsiniz. Neticede kabuk çalıştırılabilir dosyaları nerelerde araması gerektiğini PATH değişkeninden öğreniyor.

Kabuğa bir komut girdiğimizde, kabuğun bu komut ile eşleşen dosyayı PATH yolunda aradığını ve bulabilirse çalıştırdığını kanıtlamak için hemen basit bir test yapabiliriz.

Test edebilmek için PATH yolundaki herhangi bir dizinde bulunmayan ve çalıştırıldığını bize kanıtlayabilecek bir programa ihtiyacımız var.

Bunun için kabuğa girdiğimiz komutları bir dosyaya kaydedip, komutların bu dosyadan çalıştırılmasını sağlayan bir betik dosyası yani basit bir program oluşturabiliriz. Zaten kabuğun programlanabilir olduğundan daha önce de çok kısaca bahsetmiştim. Kabuk aynı zamanda programlanabilir olduğu için, komutları dosya içine yerleştirip kendi amaçlarımıza uygun betik dosyaları oluşturabiliyoruz. Hemen canlı örneğini görmek için kendimize bir kabuk programı yazalım.

Ben, çalıştığında konsola "**Program Çalıştı!**" ifadesini basacak çok basit bir betik dosyası oluşturmak istiyorum. Tüm işlemleri de komut satırı üzerinden yapacağım. Ancak merak etmeyin burada kullandığım tüm komutları ileride ayrıca ele alıyor olacağız.(Sürekli sonra ele alacağız diyorum, ancak tüm konular birbiri ile bağlı olduğu için böyle olmak zorunda.) Sizin şimdilik sadece beni takip etmeniz yeterli.

Öncelikle içerisinde komutları girebileceğim bir betik dosyası oluşturmak istiyorum. Bunun için konsola `cat > betik.sh` şeklinde yazıp komutumuzu girebiliriz.

```
└─(taylan@linuxdersleri) - [~]
└─$ cat > betik.sh
```

Buradaki `cat` komutunun ardından kullandığımız `>` operatörü ile "betik.sh" isimli bir dosya oluşturulmasını ve bu dosyaya konsoldan yeni veriler gireceğimizi belirtmiş olduk. Konsol da bir alt satıra geçip bizden dosyaya

yazılacak verileri beklemeye başladı.

Şimdi buraya yazacağımız verileri betik dosyasına kaydetmemiz mümkün. Yani bir nevi komut satırı üzerinden not defteri özelliği gibi düşünebilirsiniz. Ben çalıştırıldığı zaman konsola “**Program Çalıştı**” ifadesini bastırarak bir kabuk programı oluşturmak istiyorum. Bunun için de buraya `echo "Program Çalıştı"` şeklinde komutumu ekliyorum.

```
(taylan@linuxdersleri)-[~]  
└─$ cat > betik.sh  
echo "Program Çalıştı"
```

Şimdi artık buraya yazdığım komutun bu dosyaya kaydolması için, yazdıklarımın bittiğini haber vermem gerekiyor. Bunun için de `Ctrl + D` tuşlaması yapmam yeterli. Bu sayede `cat` komutu veri girişinin tamamlandığını anlayıp, “betik.sh” isimli dosyaya girdiğim verileri kaydedecek. Hatta kontrol etmek için `cat betik.sh` komutu ile dosyanın içeriğini konsola bastırabiliriz.

```
(taylan@linuxdersleri)-[~]  
└─$ cat betik.sh  
echo "Program Çalıştı"  
(taylan@linuxdersleri)-[~]  
└─$
```

Bakın yazmış olduğum komut bu dosyaya kaydedilmiş.

Böylelikle istediğim amaca uygun bir betik dosyası oluşturmuş oldum. Bu dosyanın çalıştırılabilmesi için son olarak çalıştırma yetkisini de vermemiz gerekiyor. Eğer dosyanın çalıştırma yetkisi yoksa dosya bulunsada çalıştırılmaz. Bu durumu teyit etmek için `./betik.sh` komutu ile betik dosyasını çalıştırmayı deneyebiliriz.

```
└─$ ./betik  
bash: ./betik: Permission denied
```

Bakın gördüğünüz gibi yetki hatası aldık çünkü bu dosyanın henüz çalıştırılma yetkisi yok. Şimdi bu sorunu aşmak için dosyamıza `chmod +x betik.sh` komutu ile çalıştırma yetkisi verelim. Buradaki `chmod` komutu okuma yazma ve çalıştırma gibi yetkilerin yönetimi için kullandığımız bir araç.

Neticede bu komutumuzla birlikte artık “betik.sh” dosyasına çalıştırma yetkisini de kazandırmış olduk. Şimdi teyit etmek için tekrar `./betik.sh` şeklinde komutumuzu girelim.

```
└─$ ./betik.sh  
Program Çalıştı !
```

Gördüğünüz gibi bu kez betik dosyası sorunsuzca çalıştı ve konsola “**Program Çalıştı**” ifadesini bastırdı. Artık çalıştırılabilir bir program dosyamız olduğuna göre ve çalıştığını da bizzat teyit ettiğimize göre PATH yolunun

amacını uygulamalı olarak test edebiliriz.

Ayrıca burada benim dosyayı çalıştırmak için kullandığım `./betik.sh` komutunu merak etmiş olabilirsiniz. Kabuğa çalıştırılacak dosyayı tam konumu ile verdiğinizde kabuk o dosyanın uygun ortamda çalıştırılmasını sağlıyor. Normalde benim girdiğim `./betik.sh` komutu, çalıştırılacak dosyanın tam konumunu belirtiyor. Betik dosyasını şu an kabuğun çalışmakta olduğu dizinde oluşturduğum için, bulunduğum dizini `.` **nokta** ile belirtip `/` **slash** işaretinin ardından dosyanın ismini yazdığımda kabuk benim `"betik.sh"` dosyasını çalıştırmak istediğimi anlıyor ve çalıştırıyor. Yani harici olarak PATH yoluna bakmasına gerek kalmıyor çünkü biz doğrudan çalıştırılacak dosyanın konumunu kabuğa bildirmiş oluyoruz.

Neticede kabuğun bir dosyayı çalıştırabilmesi için o dosyanın tam konumunu biliyor olması gerekiyor. Eğer bu dosya kabuğun çalışmakta olduğu mevcut dizinde değilse dosyanın tam dizin adresini uzun uzadıya yazmamız gerekir.

Bu durumu gözlemlemek için dosyamızı farklı bir dizine taşıyalım. Taşıma işlemini komut satırından yapmak istersek İngilizce "move" yani "taşınma" ifadesinin kısaltması olan `mv` komutunu kullanabiliriz. Dosyayı taşımak için `mv` komutundan sonra taşımak istediğimiz dosyanın tam adresini ve nereye taşımak istediğimizi belirtiyoruz. Betik.sh dosyasını Downloads klasörüne taşıyacağım için ilk olarak betik dosyasının adresini bu şekilde yazdım. Daha sonra dosyanın taşınmasını istediğim dizini de ikinci argüman olarak girdim. Yani taşımak için `mv betik.sh ~/Downloads/` komutunu girip **Downloads** dizini altına taşıyacağım.

```
(taylan@linuxdersleri)-[~]  
$ mv betik.sh ~/Downloads/
```

Konumu değiştiği için konsoldan bu dosyaya ulaşmak için artık `./Downloads/betik.sh` şeklinde komut girmem gerekecek. Çünkü betik dosyası kabuğun bulunduğu ev dizini altındaki **Downloads** klasörü içinde yer alıyor.

```
(taylan@linuxdersleri)-[~]  
$ ./Downloads/betik.sh  
Program Çalıştı
```

Tabii ki her seferinde dosyaların konumunu hatırlamak ve bu şekilde uzun uzadıya yazmak pek de verimli değil. Çünkü sık kullandığımız bunun gibi pek çok farklı konumda dosyamız olabilir. Bunun yerine çalıştırılacak dosyayı **PATH** yolu üzerinde yer alan bir dizin altına taşırsak, kabuğa yalnızca dosyanın adını vererek istediğimiz konumdan dosyayı çalıştırabiliriz.

Ben bu durumu teyit etmek için **PATH** değişkeninde tanımlı olan dizinlerden birine betik dosyamı taşımak istiyorum. PATH değişkeninde tanımlı olan herhangi bir dizin içine taşıyabiliriz. Ben örnek olarak **/usr/local/bin** dizini altına taşımak istiyorum.

Fakat PATH yolu üzerinde yer alan dizinler kabuk tarafından taranıp buradaki dosyalar çalıştırıldığı için yetkisi olmayan kullanıcılar buraya yeni dosya ekleyemezler. Sadece yetkisi olan kullanıcılar buraya yeni dosya ekleyebilir. Denemek için `mv ~/Downloads/betik.sh /usr/local/bin` komutunu girebiliriz.

```
└─$ mv ~/Downloads/betik.sh /usr/local/bin
mv: cannot move 'betik.sh' to '/usr/local/bin/betik.sh': Permission denied
```

Neticede gördüğünüz gibi girmiş olduğumuz komut doğru olsa da yetki hatası aldık. Çünkü taşıma işlemini yapmak için yetkili olduğumuzu kanıtlamadık.

Yetkili olduğumuzu kanıtlamak için komutun başına **sudo** eklememiz ve mevcut hesabımızın parolasını girmemiz gerekiyor. Komutumuzu bu şekilde düzenleyip tekrar girelim.

```
└─$ sudo mv ~/Downloads/betik.sh /usr/local/bin
[sudo] password for taylan:
```

Bakın benden parola girmemi istiyor, hesabımın parolasını girip onayladığımda dosya ilgili konuma sorunsuzca taşınmış olacak.

Artık bu sayede kabuk hangi konumda çalışıyor olursa olsun betik.sh komutunu girdiğimizde, betik dosyamız çalışacak. Çünkü biz **betik.sh** ifadesini girdiğimizde kabuk öncelikle yerleşik komutlara bakacak ve burada olmadığını fark edince **PATH** dizinindeki tüm klasörlere sırasıyla bakarak bu isimle eşleşen dosyayı arayacak. Neticede bizim eklemiş olduğumuz dosyayı bulacak ve çalıştıracak. Yani artık özellikle dosyanın konumunu belirtmemiz gerekmiyor. Hadi hemen denemek için konsola **betik.sh** şeklinde yazalım.

```
└─$ betik.sh
Program Çalıştı !
```

Bakın betik dosyası şu an kabuğun çalıştığı dizinde bulunmuyor olmasına rağmen yalnızca ismini girerek dosyanın çalıştırılmasını sağlayabildik. Hatta derseniz daha somut bir teyit olması için grafiksel arayüzden başka bir dizine gidip burada sağ tıklayıp yeni bir konsol açıp **betik.sh** komutunu burada girmeyi deneyebilirsiniz. Örneğin ben masaüstüne sağ tıklayıp konsolumu açtım ve **betik.sh** komutunu girdim.

```
└─(taylan@linuxdersleri) - [~/Desktop]
└─$ betik.sh
Program Çalıştı!
```

İşte bizzat teyit ettiğimiz gibi bu örnek, kabuğun harici bir programı çalıştırmak için **PATH** olarak geçen dizinlere baktığını kanıtlıyor. Sizler de bu şekilde, kabuk üzerinden ismiyle çağırıp çalıştırmak istediğiniz programlarınızı **PATH** dizinlerinden birine taşıyabilirsiniz. Ayrıca dilersek PATH üzerinde yer alan dizinlere yeni bir dizin daha ekleyebiliriz. Bu sayede eklediğimiz dizin içinde yer alan dosyalar bash kabuğu tarafından çalıştırılacak dosya ismi olarak görülebilirler.

PATH Yoluna Yeni Dizin Ekleme

Anlatımla başlamadan önce PATH yoluna yeni bir dizin eklemenin güvenlik açısından pek önerilen bir işlem olmadığını belirtmek istiyorum. Güvenli değil çünkü yeni eklediğiniz dizin adresi için gereken yetkilendirme ve

sıkılaştırma önlemlerini almamış olabiliyoruz. Varsayılan olarak tanımlı olan PATH adreslerinde ise zaten yetkilendirme ayarları yapılmış oluyor. Hatırlarsanız zaten betik dosyamızı taşımak için `sudo` komutu ile yetkili olduğumuzu kanıtlamamız gerekmişti. Yani varsayılan PATH dizinlerinin yalnızca yetkili kişilerce düzenlenebilecek şekilde sıkılaştırıldığını bizzat deneyimledik. Bizim sonradan ekleyeceğimiz dizinin yetki ayarları doğru şekilde tanımlı olmazsa bu dizine yetkisiz kullanıcılar da dosya taşıyabilir ve kabuğun bu dosyaları da çalıştırmasını sağlayabilir. Bu durum elbette güvenlik riski demek oluyor.

Dolayısıyla varsayılan olarak tanımlı olan PATH adreslerini kullanmanız çok daha doğru ve güvenli bir yaklaşımdır. Yine de ihtiyaç duymanız halinde kullanabilmeniz, ve kabuğun çalışma yapısını daha iyi kavrayabilmeniz için kısaca PATH yoluna nasıl yeni izin ekleyebileceğimize de değinmek istiyorum.

Bildiğiniz gibi PATH yolu üzerindeki dizinlerin hangileri olduğunu öğrenmek için `echo $PATH` komutunu kullanıyoruz. Hatta kullandığımız bu komutun anlamını özellikle çok kısaca da olsa açıklamıştık hatırlıyorsanız. `echo` komutu, kendisine argüman olarak verilen değerleri çıktı olarak bastırıyor. Buradaki `$PATH` ifadesi ise bir değişkendir. Eğer programlama geçmişiniz varsa zaten değişkenlerin ne olduğunu mutlaka biliyorsunuzdur. Değişkenler, tanımlı olan değerlere tekrar tekrar tek bir değişken ismi üzerinden ulaşılabilmesini sağlayan yapılardır. Bash kabuğunda da bizzat daha önce de gördüğümüz SHELL ve PATH gibi değişkenler olduğunu zaten biliyoruz. Eğer biz PATH yolunu değiştirmek istiyorsak, PATH değişkeninde tanımlı olan dizinleri yani PATH değişkeninin değerini yeniden düzenleyebiliriz. Ancak yeni bir PATH dizini eklemekten önce bash kabuğunda değişkenlerin nasıl çalıştığına temel olarak değinmek istiyorum. Bu sayede gerçekleştireceğimiz işlemleri çok daha bilinçli şekilde yerine getiriyor olacağız.

Değişkenler

Bash kabuğunda değişken tanımlamak çok kolay. Ben basit bir örnek olması için `ad=taylan` şeklinde yazıp `ad` isimli değişkene “**taylan**” değerini tanımlıyorum.

```
(taylan@linuxdersleri) - [~]  
└─$ ad=taylan  
(taylan@linuxdersleri) - [~]  
└─$
```

Bu sayede ne zaman `ad` isimli değişkeni çağırarak olursam “taylan” verisine ulaşabiliyor olacağım. Hemen denemek için `echo $ad` komutuyla değişkenin değerini konsola bastıralım.

```
(taylan@linuxdersleri) - [~]  
└─$ echo $ad  
taylan
```

Bakın “taylan” çıktısını almış oldum. Çünkü `echo` aracına buradaki `ad` isimli değişkeni konsola bastırması için argüman olarak verdik, `echo` aracı da görevini yapıp bu değişkenin değerine bastırdı. Zaten daha önce **PATH** ve **SHELL** değişkenlerini de bu şekilde konsola bastırmıştık. Değişkenlerden önce dolar işareti koyduğumuzda `echo` bu ifadenin değişken olduğunu anlıyor.

tanımlamak için: `değişken-ismi=değişken-değeri`

bastırmak için: `echo $değişken-ismi`

İşte en yalın haliyle bir değişken tanımlamak ve değişkenin değerine görüntülemek bu şekilde. Değişkenler ile ilgili diğer detaylara girmeyeceğiz fakat çok ufak bir detaydan haberdar olmamız gerekir. Bu detay da değişkenin alt kabuklar üzerinde de geçerli olabilmesini sağlayan `export` komutu.

`export` Komutu | Global Değişkenler

Eğer biz tanımlamış olduğumuz değişkeni `export` ile global hale getirmezsek, mevcut kabuğun altında başlatılan diğer kabuklar üzerinden bu değişken değerine ulaşamıyoruz. Bu durumu daha iyi anlamak için mevcut konsolumuza `bash` komutunu girip yeni bir kabuk başlatalım.

```
(taylan@linuxdersleri) - [~]  
└─$ bash  
(taylan@linuxdersleri) - [~]  
└─$
```

Bakın ben “`bash`” komutunu girerek şu anda yeni bir kabuk başlatıp bu kabuğa geçiş yapmış oldum. Yani şu an gireceğim tüm komutlar bu yeni kabuk tarafından işlenecek. Teyit etmek için biraz önce tanımladığım `ad` isimli değişkeni burada da bastırmayı deneyebiliriz. Bunun için tekrar `echo $ad` şeklinde komutumuzu girelim.

```
(taylan@linuxdersleri) - [~]  
└─$ echo $ad  
  
(taylan@linuxdersleri) - [~]  
└─$
```

Bakın herhangi bir çıktı almadık çünkü ben bu `ad` değişkenini `export` komutu ile global hale getirmemiştim. Dolayısıyla yeni açtığım ve şu anda emirler verdiğim bu kabuğun da bu değişkenden haberi yok.

Şimdi bu yeni başlattığımız kabuğu `exit` komutu ile kapatıp bir önceki kabuğa dönelim.

```
(taylan@linuxdersleri) - [~]  
└─$ exit  
exit  
  
(taylan@linuxdersleri) - [~]  
└─$
```

`exit` komutu sayesinde yeni başlattığımız kabuk kapatıldı, emin olmak için yine `ad` isimli değişkeni `echo $ad` komutu ile bastırmayı deneyebiliriz.

```
(taylan@linuxdersleri)-[~]  
└─$ echo $ad  
taylan  
  
(taylan@linuxdersleri)-[~]  
└─$
```

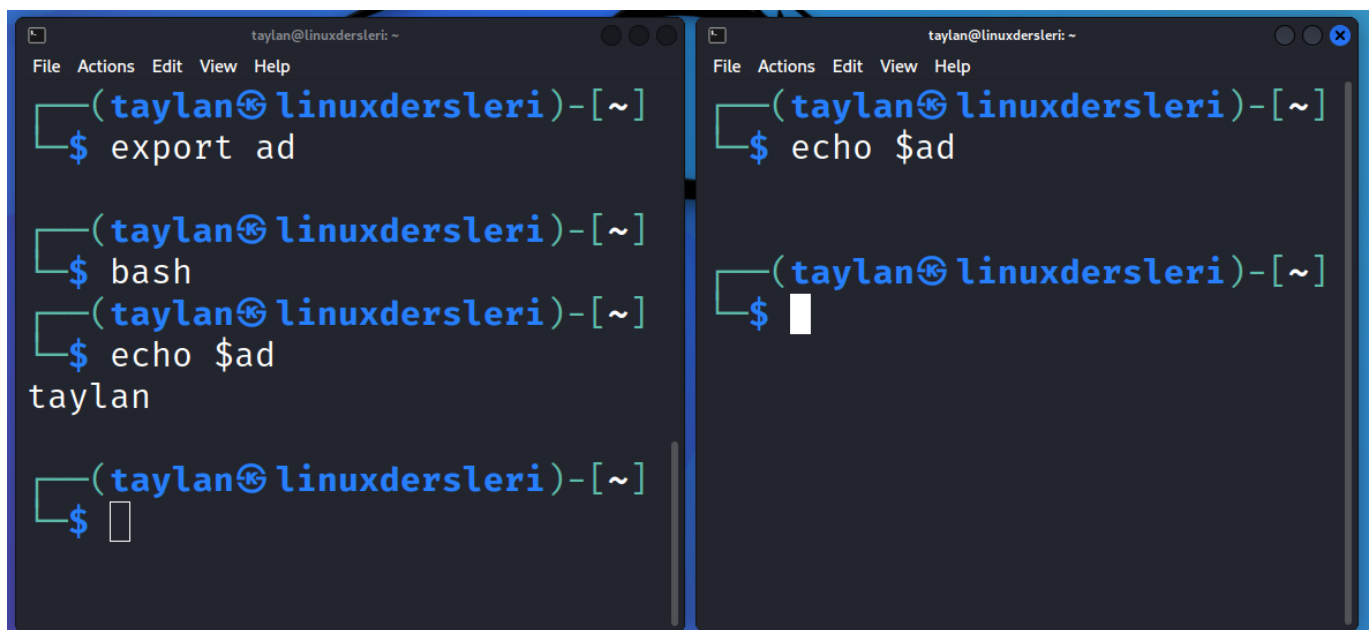
Bakın bu kez sorunsuzca bastırabildim çünkü bu değişkeni zaten bu kabukta tanımlamıştım. Şimdi **ad** isimli değişkenin alt kabuklar tarafından da bulunabilmesi için **export ad** komutu ile global hale getirelim ve tekrar **bash** komutunu girip yeni bir alt kabuk başlatalım. Kabuk başlatıldıktan sonra da **echo \$ad** komutu ile değişkenin bu alt kabukta tanınma durumunu yani global olup olmadığını sorgulayalım.

```
(taylan@linuxdersleri)-[~]  
└─$ export ad  
  
(taylan@linuxdersleri)-[~]  
└─$ bash  
(taylan@linuxdersleri)-[~]  
└─$ echo $ad  
taylan  
  
(taylan@linuxdersleri)-[~]  
└─$
```

Bakın bu kez sorunsuzca değişken değerine ulaşabildik çünkü **export** komutu ile bu değişkeni global hale getirdik.

İyi güzel ama, ya yeni bir konsol açarsam, yani mevcut kabuk altında yeni bir kabuk başlatmaktansa yeni bir bağımsız konsoldaki kabuğu kullanırsam ne olur ?

Denemek için yeni bir konsol penceresi açıp **echo \$ad** komutu ile değişkeni sorgulayabiliriz.



Yeni açtığımız konsolda hiç bir çıktı alamadık. Şimdi bir de bu konsol üzerinden **PATH** değişkenini bastırmayı deneyelim.



The image shows two terminal windows side-by-side. Both windows have a title bar that says 'taylan@linuxdersleri: ~'. The left window shows the command 'echo \$ad' being executed, resulting in the output 'taylan'. The right window shows the command 'echo \$PATH' being executed, resulting in the output '/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games'. Both windows have a menu bar with 'File', 'Actions', 'Edit', 'View', and 'Help'.

Gördüğünüz gibi her iki konsolda da **PATH** değişken değeri basıldı. Peki ama nasıl oluyor da bizim **export** komutu ile global hale getirdiğimiz değişkene başka bir konsoldan ulaşamıyorken, **PATH** değişkenine tüm konsollardan ulaşabiliyoruz ?

Bu durumun sebebi miras yapısıdır. Mevcut kabuk yalnızca kendisinin başlatmış olduğu yeni işlemlere değişken gibi değerleri miras bırakabiliyor. Biz yeni bir konsol penceresi açtığımızda, halihazırda çalışmakta olan kabuklardan bağımsız yeni bir kabuk bu konsolda başlatılıyor. Dolayısıyla bağımsız bir kabuk tarafından **export** edilen değişken, bir diğer bağımsız kabuk tarafından miras alınamıyor. Çünkü arasında değişken aktarımını gerektirecek bir miras bağı bulunmuyor. Mevcut kabuk üzerinden **bash** komutu ile yeni kabuk başlattığımızdaysa, mevcut kabuk bu işlemi kendisi başlattığı için değişkenlerini yeni kabuğa miras olarak aktarabiliyor. Tüm meselenin özeti aslında bu.

Peki ama **PATH** değişkenine nasıl tüm konsollardan yani tüm bağımsız kabuklardan ortak olarak ulaşabiliyoruz ?

Bu durumun sebebi, **PATH** değişkeninin kabuk başlatılırken kabuk tarafından okunan konfigürasyon dosyalarından birinde tanımlanmış olması.

Eğer biz de değişkenlerimizi kabuk tarafından okunan bu konfigürasyon dosyalarından birine eklersek, değişkenimiz her yeni başlatılan kabuk tarafından okunacağı için tüm kabuklardan bu değişkene ortak şekilde erişebileceğiz. Peki bu konfigürasyon dosyaları hangi dosyalar diye soracak olursanız:

Bash kabuğu konfigürasyonlar için temelde iki tür dosyayı okuyor. Bunlar; sistem genelinde **tüm kullanıcılar için geçerli olan** ve **spesifik kullanıcıya özel olan** iki farklı türdeki konfigürasyon dosyalarıdır.

Konfigürasyon Dosyaları

Sistem Geneli İçin Yapılandırma

Linux çok kullanıcıli bir işletim sistemi olduđu için tüm kullanıcılar üzerinde geçerli olabilecek toplu yapılandırma kuralları tanımlayabilmek adına sistem genelinde kullanılan yapılandırma dosyalarında düzenlemeler yapabiliriz.

- `/etc/profile`
- `/etc/bashrc`
- `/etc/bash.bashrc`

Kullanıcı Bazlı Yapılandırma

Değişikliklerin tüm kullanıcıları değil de özel olarak tek bir kullanıcıyı etkilemesini istersek, kullanıcının kendi ev dizininde bulunan yapılandırma dosyalarında düzenlemeler yapabiliriz.

- `~/.bash_profile`
- `~/.bashrc`
- `~/.bash_login`
- `~/.profile`

İşte ihtiyaçlarımıza göre hangi kapsamda değişiklik yapmak istiyorsak ona uygun dosyalarda düzenleme yapmamız gerekiyor.

Ben hepsini listeledim ancak tabii ki tüm sistemlerde saydığım tüm bu dosyalar varsayılan olarak bulunmuyor olabilir. Tek yapmanız gereken benim saymış olduğum dosyalardan hangisi sizin sisteminizde mevcutsa o dosyada değişiklik yapmak. Çünkü farklı dağıtımlar farklı dosyaları konfigürasyon için kullanabiliyor. Ancak merak etmeyin farklılıkların doğrudan bir önemi yok. Sizin kullanmakta olduğunuz sistemde hangi yapılandırma dosyası mevcutsa onu düzenleyip kullanabilirsiniz. Zaten muhtemelen kullandığınız dağıtımda benim bahsettiğim dosyalardan biri yoksa diğeri mutlaka vardır. Biraz göz atarsanız hangisini kullanmanız gerektiğini kolayca fark edebilirsiniz.

Birden fazla dosya bulunuyor olmasının bir nedeni var ancak ben temel eğitim için bahsedip kafanızı karıştırmak istemiyorum. Buradaki dosyaların hepsinin kullanım amacı ve okunma sıralaması farklı. Eğer daha fazla detay öğrenmek isterseniz [\[buraya\]\({{ site.url }}/bash-konfigurasyon-dosyaları.html\)](#) göz atabilirsiniz.

Tekrar konumuza dönecek olursak, siz bash kabuğundaki değişiklikleri hangi düzeyde gerçekleştirmek istiyorsanız ona uygun olan konfigürasyon dosyalarından birinde düzenleme yapmanız gerekiyor. Örneğin ben sistemdeki tüm kullanıcıların PATH yolunu düzenlemek istersem, benim kullanmakta olduğum dağıtımda mevcut bulunan ve tüm kullanıcılar için geçerli olan **`/etc/bash.bashrc`** dosyasında düzenleme yapabilirim. Ya da spesifik olarak yalnızca kendi kullanıcı hesabım için PATH yolunu değiştirmek istersem bu değişikliği kendi ev dizinimdeki yani örneğin **`/home/taylan`** dizini altındaki **`.bashrc`** dosyasında yapabilirim.

Bu kadar açıklama yeter. Bizzat uygulayarak bahsetmiş olduklarımızın sonuçlarını gözlemleyelim. Örnek olarak PATH yoluna yeni bir dizin adresi eklemeyi deneyebiliriz.

PATH Yoluna Yeni Dizin Ekleme

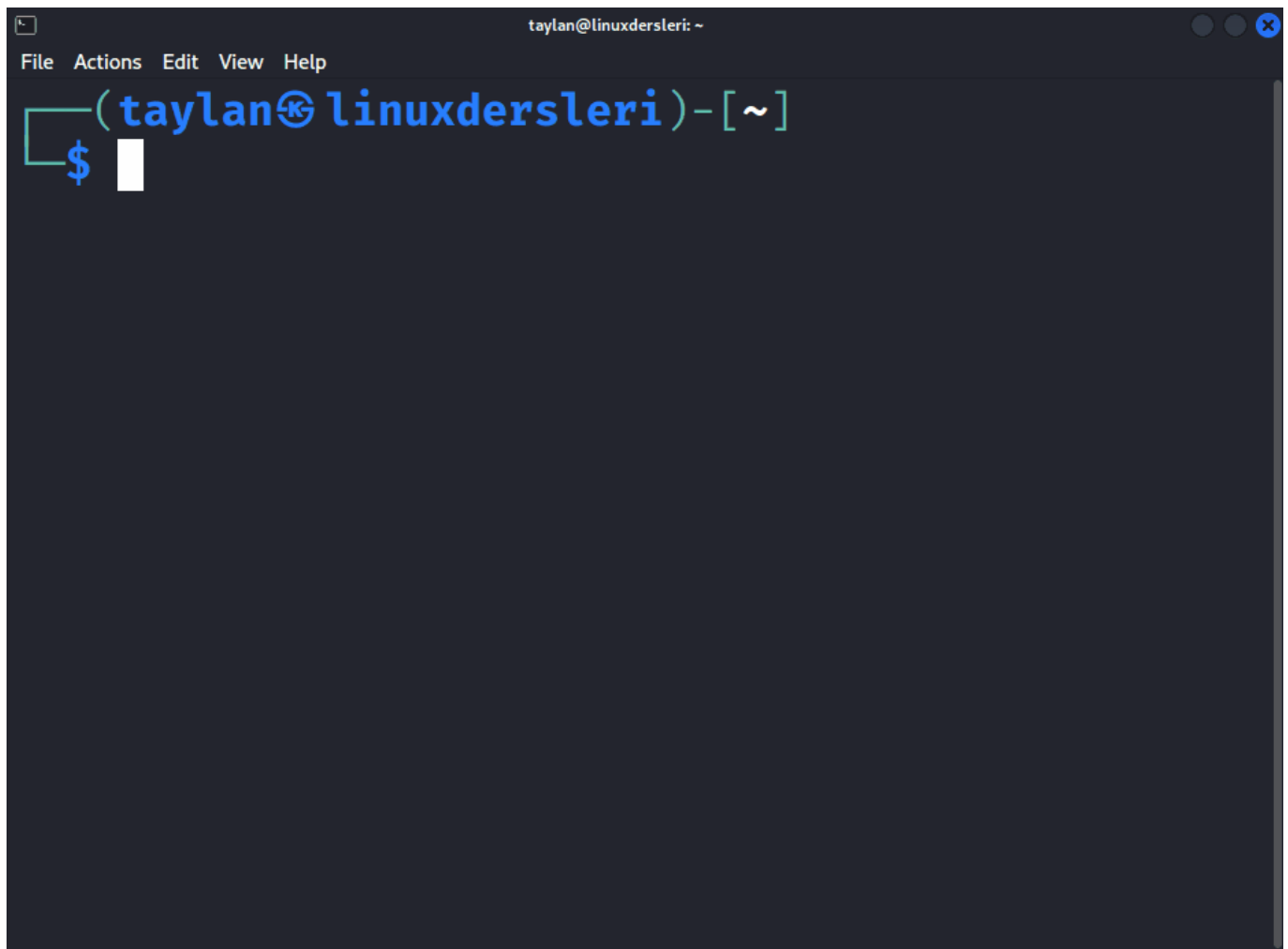
Ben PATH yolundaki değişikliğin tüm sistem genelinde yani tüm kullanıcılar üzerinde ortak olarak etkili olmasını istediğim için **`/etc/bash.bashrc`** dosyasında değişiklik yapacağım.

Öncelikle ekleyeceğimiz yeni dizini oluşturmak üzere `mkdir ~/Desktop/yeni-dizin` komutu ile masaüstü dizinimizde ***"yeni-dizin"*** isimli klasörümüzü oluşturalım.

```
(taylan@linuxdersleri)-[~]  
$ mkdir ~/Desktop/yeni-klasor
```

Şimdi bu klasörün tam dizin adresini PATH yoluna ekleyebiliriz. Eklemek için `sudo nano /etc/bash.bashrc` komutu ile konfigürasyon dosyasını açalım. `sudo` komutu parola girmenizi isteyecektir, hesabınızın parolasını girip `enter` tuşu ile onaylayın.

```
(taylan@linuxdersleri)-[~]  
$ sudo nano /etc/bash.bashrc  
[sudo] password for taylan:
```



i Not: Burada dosyanın ismini nokta da dahil eksiksiz yazdığınızdan emin olun. Eğer doğru yazmazsanız veya gereksiz yere boşluk bırakırsanız dosya açılmaz çünkü ilgili dosya bulunamaz.

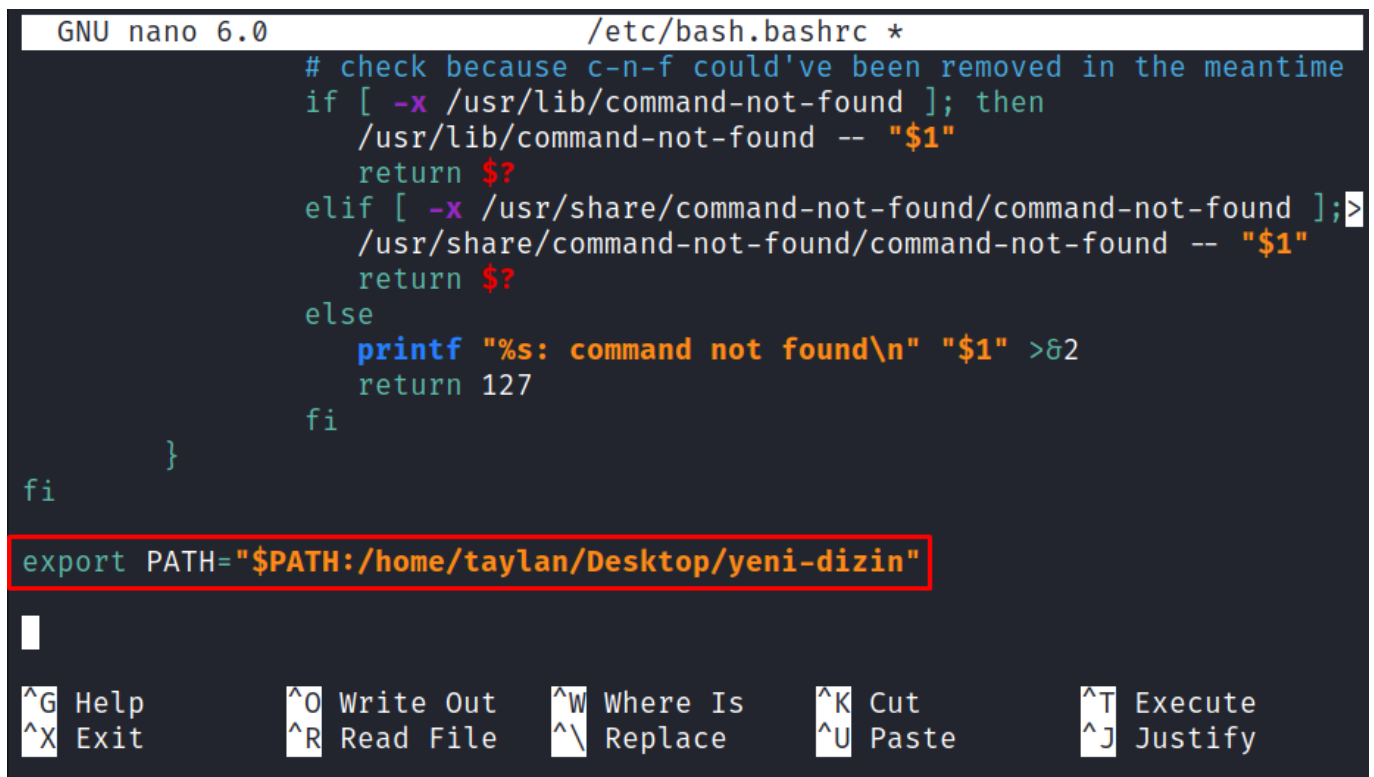
Açılmış olan bu dosya içerisine **PATH** değişkeninin değeri olarak yeni oluşturduğumuz dizini de eklememiz gerekiyor. Bu noktada eklemek istediğiniz dizinin tam adresini doğru şekilde girmeniz şart. Örneğin ben taylan kullanıcısının ev dizini altındaki **Desktop** klasörü içerisinde “yeni-dizin” isimli klasörü eklemek istediğim için tam olarak “**/home/taylan/Desktop/yeni-dizin**” dizinini belirtmem gerek. Siz de kendi dizininize göre bu adresi belirtmelisiniz. Eğer eklemek istediğiniz dizinin tam konumunu bilmiyorsanız ilgili dizindeyken sağ

tıklayıp "konsolu burada başlat" seçeneği ile konsolu açın ve `pwd` komutunu girip mevcut dizin adresini öğrenin.

```
(taylan@linuxdersleri) - [~/Desktop/yeni-dizin]
$ pwd
/home/taylan/Desktop/yeni-dizin
```

Dizin adresinden emin olduktan sonra `export PATH="$PATH:yeni-dizin-adresi"` şeklinde **PATH** yoluna eklemek istediğiniz yeni dizini tanımlamanız gerek. Örneğin ben aşağıdaki şekilde tanımlıyorum.

```
export PATH="$PATH:/home/taylan/Desktop/yeni-dizin/"
```



```
GNU nano 6.0 /etc/bash.bashrc *
# check because c-n-f could've been removed in the meantime
if [ -x /usr/lib/command-not-found ]; then
    /usr/lib/command-not-found -- "$1"
    return $?
elif [ -x /usr/share/command-not-found/command-not-found ];>
    /usr/share/command-not-found/command-not-found -- "$1"
    return $?
else
    printf "%s: command not found\n" "$1" >&2
    return 127
fi
}
fi

export PATH="$PATH:/home/taylan/Desktop/yeni-dizin"

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

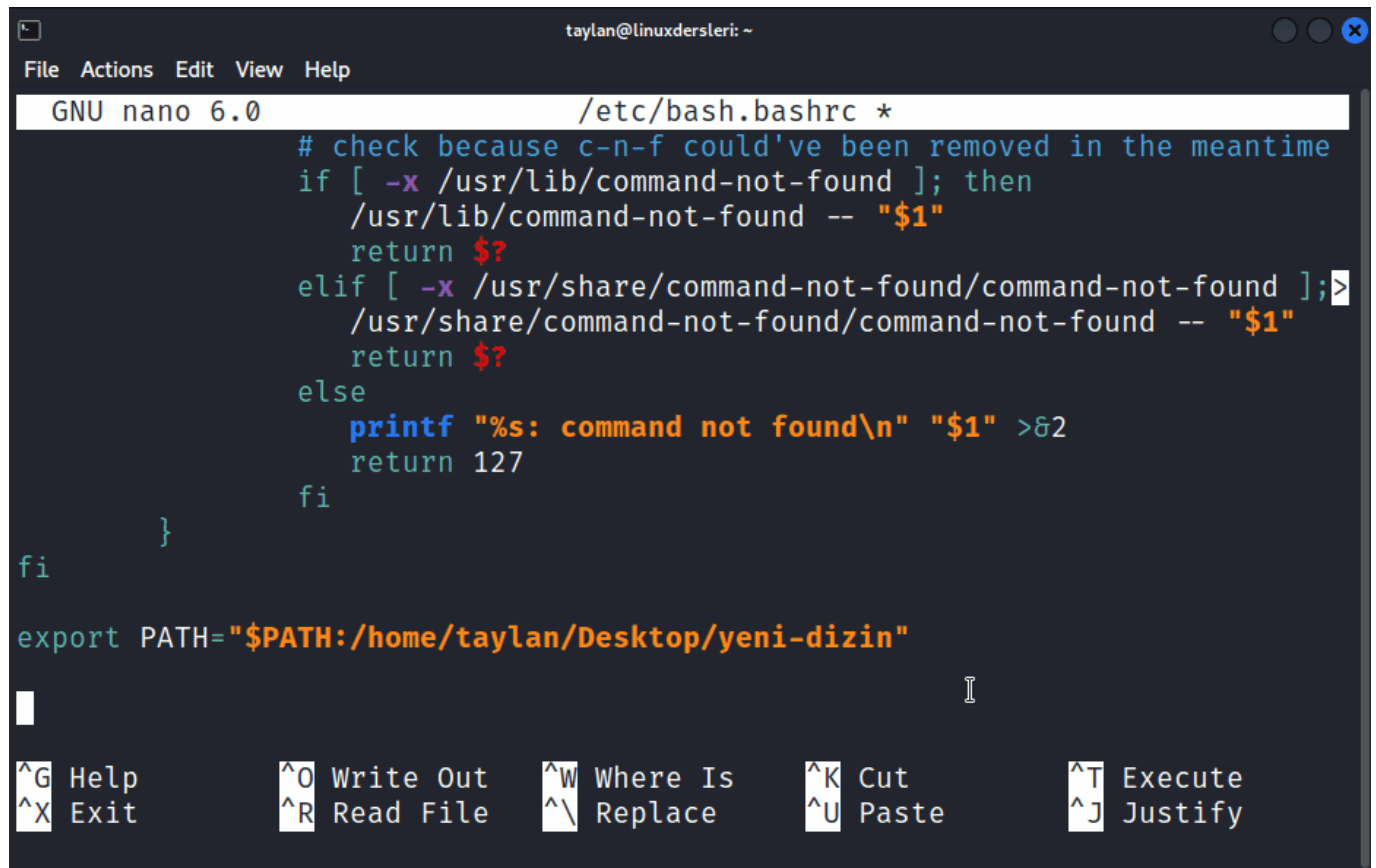
Tanımlamayı açıklayacak olursak buradaki `export` komutu bildiğiniz gibi kendisinden sonraki değişkeni global hale getiriyor. Buradaki değişikliğin bu konfigürasyon dosyasını okuyan tüm kabuklarda geçerli olması için `export` kullandık. `export` komutundan sonra tekrar **PATH** isimli değişken tanımlayıp `$PATH` sayesinde mevcut **PATH** değişkeninin değerini koruduk. Mevcut değişken değerinden sonra iki nokta karakterinin hemen ardından da yeni dizin adresini ekledik. Bu sayede **PATH** yolundaki eski dizin adresleri korunup, yeni dizin adresi de sonuna eklenmiş oldu.

Eğer kafanıza takıldıysa, burada iki nokta üst üste işareti kullandım çünkü **PATH** yolundaki tüm dizinler birbirinden iki nokta üst üste karakteri ile ayrılıyor. Ben de mevcut **PATH** yolundaki dizinlere ek olarak yeni oluşturduğum klasörün tam dizin adresini de eklemiş oldum.

Benim kullanıcı hesabım **taylan** olduğu için benim ev dizinim de **/home/taylan/** dizini altında bulunuyor. Ben bu klasörü kendi ev dizimde oluşturduğum için de buraya **/home/taylan/Desktop/yeni-dizin** şeklinde tam olarak yazdım. Örneğin sizin kullanıcı adınız **hasan** veya **ayse** ise sizin ev dizininiz **/home/ayse** veya

`/home/hasan` şeklinde olacağı için izin adresini doğru şekilde yazdığınızdan emin olun. Aksi halde bu tanımlama geçersiz olur. Tekrar ediyorum; **PATH** yoluna eklemek için oluşturduğunuz yeni dizinin tam adresini ve klasör ismini eksiksiz şekilde girmeniz gerekiyor. Bu dizinin ne olduğundan emin olmak isterseniz grafiksel arayüzünden dosya yöneticisi yardımıyla oluşturduğunuz klasörün içine girip veya burada konsol başlatıp `pwd` komutuyla dosya yolu hakkında bilgi alabilirsiniz. Zaten zor bir şey değil fakat genellikle dikkatsiz öğrenciler karışıklık yaşayabiliyor.

Tamamdır, neticede yeni dizinimizi konfigürasyon dosyasına ekledik. Şimdi bu değişikliğin kaydolması için bu konfigürasyon dosyasını kaydetmemiz gerekiyor. Daha önce de yaptığımız gibi nano aracından çıkıp dosyayı kaydetmek için `Ctrl + X` tuşlaması yapıp, kayıt için `Y` tuşu ile onay vermemiz ve `enter` ile dosyayı kapatmamız yeterli.



```
GNU nano 6.0 /etc/bash.bashrc *
# check because c-n-f could've been removed in the meantime
if [ -x /usr/lib/command-not-found ]; then
    /usr/lib/command-not-found -- "$1"
    return $?
elif [ -x /usr/share/command-not-found/command-not-found ];>
    /usr/share/command-not-found/command-not-found -- "$1"
    return $?
else
    printf "%s: command not found\n" "$1" >&2
    return 127
fi
}
fi

export PATH="$PATH:/home/taylan/Desktop/yeni-dizin"

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Dosyamızı kaydettikten sonra değişikliğin geçerli olması için oturumumuzu kapatıp tekrar açmamız gerek çünkü bu dosya biz oturum açarken okunan bir konfigürasyon dosyası. Ayrıca eğer oturumu kapatıp açmak istemezseniz `source` komutundan sonra değişikliğin okunup geçerli olmasını istediğiniz dosyanın ismini de girebilirsiniz. Örneğin `*/etc/bash.bashrc*` dosyasında değişiklik yaptığım için `source /etc/bash.bashrc` şeklinde komut girebilirim.

```
(taylan@linuxdersleri)~$ source /etc/bash.bashrc
taylan@linuxdersleri.net:~$
```

Tamamdır `source` komutum sayesinde değişiklik anında geçerli oldu. Teyit etmek için `echo $PATH` komutu ile **PATH** değişkenini bastırabiliriz.


```
taylan@linuxdersleri.net:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games:/home/taylan/Desktop/yeni-dizin
```

Bakın yeni eklemiş olduğun dizin adresi **PATH** yolu üzerinde gözüküyor. Bu da demek oluyor ki artık bu klasör içinde bulunan tüm dosyalar, biz isimlerini kabuğa yazdığımızda kabuk tarafından bulunup çalıştırılabilir olacaklar. Denemek için hemen basit bir örnek yapalım.

Örnek için `cat > test.sh` şeklinde yeni bir dosya oluşturmak üzere komutumuzu girelim. Konsola çıktı bastırması için de `echo "bu bir test programıdır"` şeklinde yazıp `Ctrl + D` tuşlaması ile dosyanın kaydolmasını sağlayalım. Son olarak çalıştırılabilmesi için `chmod +x test.sh` komutu ile çalıştırma yetkisini de verelim. Son olarak bu dosyayı yeni oluşturduğunuz "**yeni-dizin**" dizinine taşımak için `mv test.sh ~/Desktop/yeni-dizin/` komutunu girelim.

```
taylan@linuxdersleri.net:~$ cat > test.sh
echo "bu bir test programıdır"

taylan@linuxdersleri.net:~$ chmod +x test.sh

taylan@linuxdersleri.net:~$ mv test.sh ~/Desktop/yeni-dizin/

taylan@linuxdersleri.net:~$
```

Böylelikle, test betiğimizi PATH yoluna eklemiş olduğumuz yeni dizine taşımış olduk. Yani artık konsola `test.sh` yazdığımızda bu dosya çalışıyor olacak.

```
taylan@linuxdersleri.net:~$ test.sh
bu bir test programıdır

taylan@linuxdersleri.net:~$
```

Bakın dosyanın konumunu belirtmeden yalnızca dosya ismiyle çalıştırabildim çünkü "yeni-dizin" isimli klasör, kabuğun komutları çalıştırmadan önce baktığı **PATH** yoluna ekli ve bu betik dosyası da bu dizin içinde. Tabii ki bu pek güvenli bir yöntem olmadığı için varsayılan **PATH** yoluna yeni bir dizine eklemenizi eğer ne yaptığınızdan emin değilseniz kesinle önermiyorum.

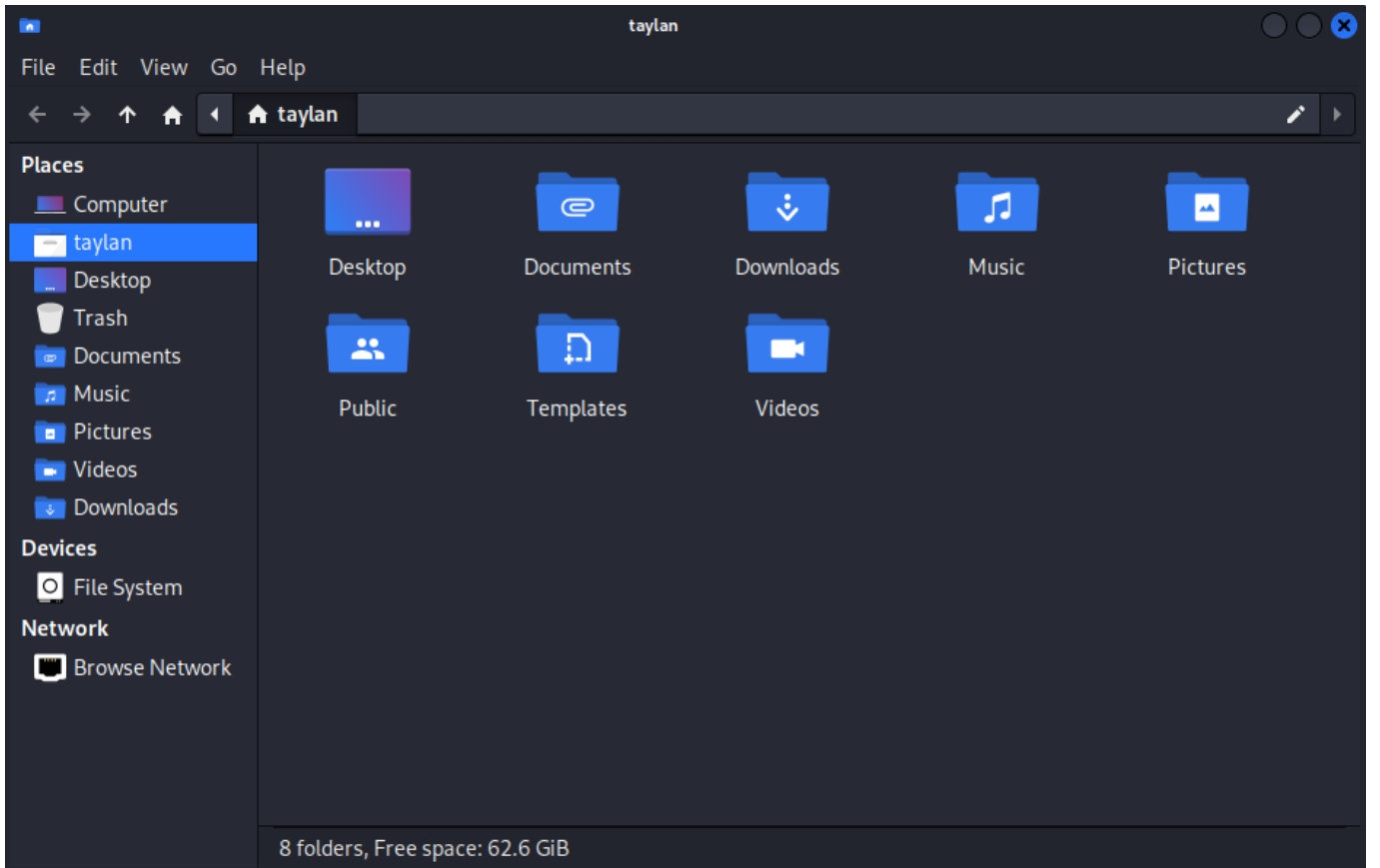
Yine de anlatımları dikkatli biçimde takip ettiyseniz **PATH** yoluna yeni dizin eklemenin ne kadar kolay olduğunu sizler de fark etmişsinizdir.

Eğer eklediğimiz yeni dizini silmek istersek değişiklik yaptığımız konfigürasyon dosyasını tekrar açıp, yaptığımız değişikliği sildikten sonra dosyayı kaydetmemiz yeterli. Tabii ki değişikliğin geçerli olması için de oturumunuzu kapatıp tekrar açabilir ya da `source` komutu ile ilgili konfigürasyon dosyasındaki değişikliklerin geçerli olmasını sağlayabilirsiniz.

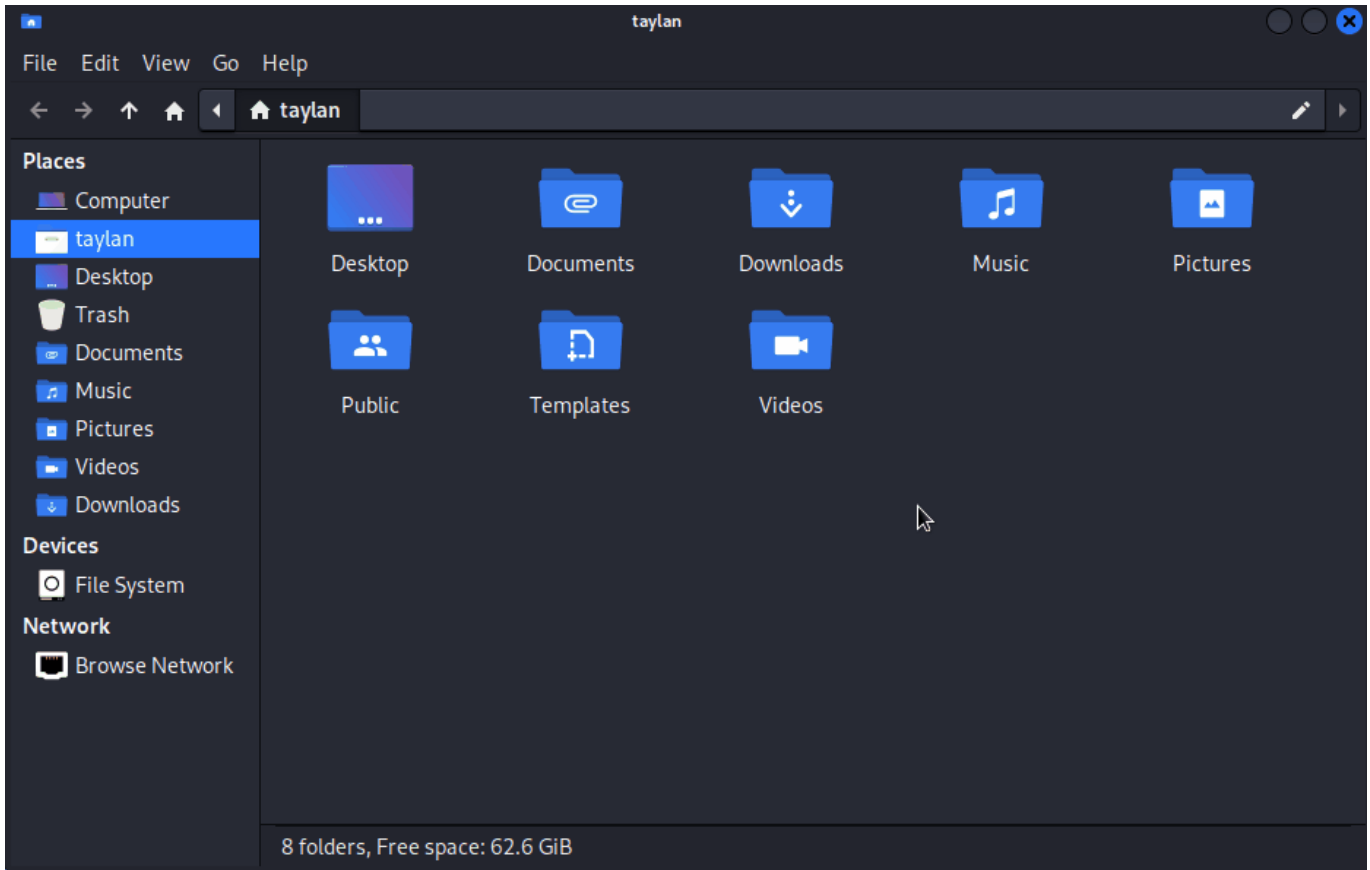
Ben örnek olması için tüm kullanıcıları etkileyen **/etc/bash.bashrc** dosyasında değişiklik yaptım. Eğer siz yalnızca tek bir kullanıcıyı etkileyecek şekilde değişiklik yapmak isterseniz kullanıcının kendi ev dizininde bulunan ilgili konfigürasyon dosyasında değişiklik yapabilirsiniz. Örneğin ben kendi kullanıcıma özel olan bir konfigürasyon tanımlamak istersem **/home/taylan/** dizinimde bulunan **.bashrc** dosyasında değişiklik yapabilirim. Aynı işlem olduğu için ben tekrar bu değişiklikten bahsetmeyeceğim fakat dediğim gibi spesifik bir kullanıcıyı etkileyecek değişiklik için o kullanıcının ev dizinindeki **.bashrc** dosyasında tıpkı burada ele aldığımız gibi değişiklik yapabilirsiniz.

Gizli Klasörler Hakkında

Hazır yeri gelmişken belirtelim, Linux sisteminde isminin başında nokta bulunan dosya ve klasörler gizli olarak sayılıyorlar. Örneğin grafiksel arayüzdeki dosya yöneticisini açtığınızda bu dosya yöneticisi varsayılan olarak ev dizininizde çalışmaya başlar. Fakat buraya göz atacak olursanız burada **.bashrc** isimli bir dosya göremezsiniz. Çünkü bu dosya isminin başındaki nokta karakteri bu dosyanın gizli olmasını sağlıyor.



Grafiksel arayüzden gizli dosyaları görmek için dosya yöneticisinin ayarlarına göz atıp gizli dosyaları gösterme özelliğini aktifleştirebiliriz. Biraz kurcalarsanız sizin kullandığınız dosya yöneticisinde nasıl yapabileceğinizi kolaylıkla keşfedebilirsiniz. Gizli dosyaları nasıl görünür kılacağınızı kendiniz keşfetmelisiniz çünkü bu ayarın konumu, kullandığınız dosya yöneticisine göre değişiklik gösterecektir.



Bakın gizli olanları gösterme özelliğini aktifleştirdiğimde, aslında ev dizinimde gizli dosya ve klasörler olduğunu grafiksel arayüz üzerinden de görebiliyorum. Hatta emin olmak istersek burada isminin başında nokta karakteri olacak şekilde yeni bir klasör ve yeni bir dosya oluşturmayı da deneyebilirsiniz. Dosya ve klasörleri oluşturduktan sonra gizlilik ayarlarını değiştirerek durumlarını teyit edebilirsiniz.

Neticede bizzat deneyimlediğimiz gibi isminin başında nokta bulunan tüm dosya ve klasörler Linux üzerinde gizli statüsündedir. Bu durumu yalnızca grafiksel arayüze özel olarak da düşünmeyin. Komut satırı üzerinde de gizli dosyalar üzerinde çalışmak için komutlarımızı gizli dosya ve klasörleri etkileyecek şekilde girmemiz gerekiyor.

Bu durumu teyit etmek için yeni bir konsol açıp ls komutunu girebiliriz.

```
(taylan@linuxdersleri) - [~]
└─$ ls
Desktop      Downloads   Pictures    Templates
Documents    Music       Public      Videos

(taylan@linuxdersleri) - [~]
└─$
```

Bakın benim ev dizinimde gizli olmayan içerikler listelendi. Eğer gizli olanlar da dahil tüm içeriği listelemek istersem ls komutuma bu kez -a seçeneğini eklemem gerek.

```
(taylan@linuxdersleri) - [~]
└─$ ls -a
.
```

```
..
.bash_history
.bash_logout
.bashrc
.bashrc.original
.bashrc.save
.bashrc.save.1
.bashrc.save.2
.cache
.config
Desktop
.dmrc
Documents
Downloads
.face
.face.icon
.gizli-dosya
.gizli-klasor
.gnupg
.ICEauthority
.java
.lessshst
.local
.mozilla
Music
Pictures
.profile
Public
.swp
Templates
.vboxclient-clipboard.pid
.vboxclient-display-svg-x11.pid
.vboxclient-draganddrop.pid
.vboxclient-seamless.pid
Videos
.wget-hsts
.Xauthority
.xsession-errors
.xsession-errors.old
.zsh_history
.zshrc

(taylan@linuxdersleri) - [~]
└─$
```

İşte tıpkı burada ele aldığımız `ls` komutu gibi, komut girerken gizli içerikleri de dahil etmek için komutlarımızın uygun seçeneklerini kullanmamız gerekiyor.

Tabii ki gizlilik özelliği çoğunlukla önemli dosya ve klasörleri korumak adına kullanılan bir özellik. Örneğin `.bashrc` dosyası önemli bir konfigürasyon dosyası olduğu için bu dosyayı gizleyerek yanlışlıkla silinmesi veya üzerinde işlem yapılması gibi olası hatalar da önlemiş oluyor. Zaten bu sebeple istisnalar hariç neredeyse hiç

bir araç `ls` komutunda olduğu gibi biz özellikle belirtmediğimiz sürece gizli olan dosya ve klasörleri işleme dahil etmezler. Bu sayede gizli içeriklerin bilinçsizce zarar görmesi de engellenmiş oluyor.

Tekrar ana konumuza dönecek olursak size son olarak konfigürasyon dosyalarında değişiklik yaparken dikkatli olmanız gerektiğini söylemek istiyorum. Eğer değişiklik yapmanız gerekiyorsa, konfigürasyon dosyalarınızın var olan yapısını bozmadan neler eklediğinizin de farkında olarak değişiklik yapın. **Ve değişiklik yapmadan önce mutlaka mevcut dosyanın yedeğini alın.** Bu sayede hatalı konfigürasyonlardan kolayca eskisine dönüş yapabilirsiniz. Özellikle zamanla içerisine pek çok ekstra konfigürasyon tanımlaması eklenmiş dosyaların yeni eklemelerden önce mutlaka yedeklerinin alınması gerekiyor. Aksi halde kabuğun doğru şekilde çalışmamasına ve sistem güvenliğinin ihlaline sebep olabilirsiniz.

Böylelikle konsola bir komut girdiğimizde, temelde neler olduğunu öğrenmiş olduk.

Tabii ki bölümün başında da belirttiğim gibi aslında konuşulabilecek çok daha fazla ek detay bulunuyor ancak Linux'u temel seviyede kullanmak için diğer detaylar fazla gelip kafanızı karıştırabilir. Hatta belki ilk defa Linux ile tanışanlar bu bölümdeki anlatımları takip ederken biraz zorlanmış olabilirler. Eğer öyleyse size önerim bu bölümü baştan tekrar edip anladığınıza emin olduktan sonra eğitime devam etmeniz. Çünkü bu bölümde ele aldıklarımız aslında çok basit ama birbiriyle bağlantılı önemli temel kavramlardan oluşuyor. Tam olarak dikkatinizi veremediyseniz veya yorgunken takip ettiyseniz belki sıkıcı veya biraz karışık gelmiş olabilir. Tek ihtiyacınız dikkatli bir biçimde adım adım takip etmek.

Eğer şahsi olarak daha ileri araştırma yapmak isterseniz buradan öğrendiklerinize ek olarak internet üzerinde bu konu özelinde araştırma yapabilirsiniz. Ben herkese hitap edebilecek temel seviye eğitim sunmak istediğim için diğer detaylara girip kafanızı karıştırmak istemiyorum. Burada öğrendiklerimiz bu eğitimi takip edip temel seviye Linux kullanımını öğrenmek için gayet yeterli. Eğitimin devamında burada edindiğimiz temel üzerine yeni bilgilerimizi ekliyor olacağız.

Bir sonraki bölümde bize hız kazandırabilecek bazı kısayollardan bahsederek devam ediyor olacağız.